Building a Quasi-Agent

For practice, we are going to write a quasi-agent that can write Python functions based on user requirements. It isn't quite a real agent, it can't react and adapt, but it can do something useful for us.

The quasi-agent will ask the user what they want code for, write the code for the function, add documentation, and finally include test cases using the unittest framework. This exercise will help you understand how to maintain context across multiple prompts and manage the information flow between the user and the LLM. It will also help you understand the pain of trying to parse and handle the output of an LLM that is not always consistent.

Reminder

Before you start, make sure you have the LiteLLM library installed:

```
pip install litellm
```

Make sure that you have an API key for the LLM provider you plan to use. You can read about how to get an API key from OpenAI here.

Make sure that you set this key as an environment variable in your terminal before running the code. See the LiteLLM documentation for more information.

Practice Exercise

This exercise will allow you to practice programmatically sending prompts to an LLM and managing memory.

For this exercise, you should write a program that uses sequential prompts to generate **any** Python function based on user input. The program should:

1. First Prompt:
   - Ask the user what function they want to create
   - Ask the LLM to write a basic Python function based on the user's description
   - Store the response for use in subsequent prompts
   - Parse the response to separate the code from the commentary by the LLM
2. Second Prompt:
   - Pass the code generated from the first prompt
   - Ask the LLM to add comprehensive documentation including:
     - Function description
     - Parameter descriptions
     - Return value description
     - Example usage
     - Edge cases
3. Third Prompt:
   - Pass the documented code generated from the second prompt
   - Ask the LLM to add test cases using Python's unittest framework
   - Tests should cover:
     - Basic functionality
     - Edge cases
     - Error cases
     - Various input scenarios

Requirements:

- Use the LiteLLM library
- Maintain conversation context between prompts
- Print each step of the development process
- Save the final version to a Python file

If you want to practice further, try using the system message to force the LLM to always output code that has a specific style or uses particular libraries.